# The Kalman Filter

Riley Martin

## 1) The Basics

This project details the use of a Kalman filter for estimating the state of a rocket during flight using noisy sensor data and a simplified physics-based model. By simulating the first one-dimensional stage, the system tracks altitude and velocity based off of an acceleration control input and altitude as a measurement. Realistic sensor noise is used to emulate uncertainties commonly encountered in such scenarios, such as imperfect altimeters and accelerometers. The Kalman filter is then implemented to repeatedly combine model predictions with measurements, producing an optimal estimate of the rocket's state over time. This approach demonstrates how state estimation techniques can significantly improve accuracy compared to simply relying on raw sensor data. This highlights the importance of data fusion in GNC applications.

## 2) Key Matrices

### 2.1 – State Vector (x)

The state vector is the estimated state of the rocket given by the Kalman filter. It takes the form of:

$$[1] \qquad x = \begin{bmatrix} position_x \\ velocity_x \\ position_y \\ velocity_y \end{bmatrix} = \begin{bmatrix} x \\ v_x \\ y \\ v_y \end{bmatrix}$$

Initially this will be set to $\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$ and future iterations of x will be calculated as:

$$[2] \qquad x_{i+1} = Fx_i + Ba_i$$

Where F is the state transition matrix, B is the control input matrix, and a is the measured acceleration at time i.

### 2.2 – State Transition Matrix (F)

As we just covered, in equation 2 the state transition matrix is multiplied by $x_i$, in doing this we are using kinematics to estimate the state at $x_{i+1}$. F is constructed as shown below:

$$[3] \qquad F = \begin{bmatrix} 1 & dt & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & dt \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Therefore, when F is multiplied by x we are given the following equations:

$$[4] \qquad F \cdot x = \begin{bmatrix} x_{i+1} \\ v_{x_{i+1}} \\ y_{i+1} \\ v_{y_{i+1}} \end{bmatrix} = \begin{bmatrix} x_i + v_x \cdot dt \\ v_{x_i} \\ y_i + v_y \cdot dt \\ v_{y_i} \end{bmatrix}$$

Based off of these equations you can see this is a simplified model, the state transition matrix does not use acceleration to estimate the state. That is left to the control input matrix.

### 2.3 – Control Input Matrix (B)

In the state vector calculation (equation 2), we multiplied B by the current acceleration (a) we do this in order to account for the acceleration in the state estimation. B is built as shown:

$$[5] \qquad B = \begin{bmatrix} \frac{1}{2}dt^2 & 0 \\ dt & 0 \\ 0 & \frac{1}{2}dt^2 \\ 0 & dt \end{bmatrix}$$

When multiplied by a we get:

$$[6] \qquad B \cdot a = \begin{bmatrix} x_{i+1} \\ v_{x_{i+1}} \\ y_{i+1} \\ v_{y_{i+1}} \end{bmatrix} = \begin{bmatrix} \frac{1}{2}a_x \cdot dt^2 \\ a_x \cdot dt \\ \frac{1}{2}a_y \cdot dt^2 \\ a_y \cdot dt \end{bmatrix}$$

Combining the product of the control input matrix (B) and the acceleration vector (a) with the product of the state transition matrix (F) and the current state vector $(x_i)$ is how we get the predictive next step of the state vector $(x_{i+1})$.

$$[7] \qquad x_{pred} = F \cdot x + B \cdot a = \begin{bmatrix} x_{i+1} \\ v_{x_{i+1}} \\ y_{i+1} \\ v_{y_{i+1}} \end{bmatrix} = \begin{bmatrix} x_i + v_x \cdot dt + \frac{1}{2}a_x \cdot dt^2 \\ v_{x_i} + a_x \cdot dt \\ y_i + v_y \cdot dt + \frac{1}{2}a_y \cdot dt^2 \\ v_{y_i} + a_y \cdot dt \end{bmatrix}$$

### 2.4 – Covariance Matrix (P)

The covariance matrix is created by estimating the standard deviation of the error in order to tell the Kalman filter just how much error it should be expecting in the data set. The covariance matrix is a diagonal matrix consisting of the standard deviations squared. (1,1) being the standard deviation of the x position error, (2,2) being that of the velocity in x, (3,3) is the same for position in y, and (4,4) is the standard deviation of the error of the y direction velocity.

$$[8] \qquad P = \begin{bmatrix} \sigma_x^2 & 0 & 0 & 0 \\ 0 & \sigma_{v_x}^2 & 0 & 0 \\ 0 & 0 & \sigma_y^2 & 0 \\ 0 & 0 & 0 & \sigma_{v_y}^2 \end{bmatrix}$$

This is set initially guessing the standard deviation of the error and is adjusted during every iteration in order to more accurately estimate $\sigma$. During these iterations we also introduce the off diagonal as the covariance which represents how error in height and velocity is correlated. We create the future iterations of P with the following equations:

$$[9] \qquad P_{pred} = FPF^T + Q$$

$$[10] \qquad P = (I - KH)P_{pred}$$

Using these equations the covariance matrix takes the form of:

$$[11] \qquad P = \begin{bmatrix} \sigma_x^2 & cov(x, v_x) & 0 & 0 \\ cov(v_x, x) & \sigma_{v_x}^2 & 0 & 0 \\ 0 & 0 & \sigma_y^2 & cov(y, v_y) \\ 0 & 0 & cov(v_y, y) & \sigma_{v_y}^2 \end{bmatrix}$$

Using this model the covariance matrix can be interpreted as a measure of the filter's confidence, increasing when relying on predictions and decreasing when incorporating measurements.

### 2.5 – Process Noise (Q)

Process noise represents the uncertainty of your model this is not an estimate of the error in your data. For example, on this project I modeled the kinematics of the rocket in order to get an idea of the altitude and velocity versus time, however my model is very simple and doesn't consider drag or thrust variation. In reality these are happening and will affect the rocket's altitude and velocity. This is where I need to account for those discontinuities. I do so using a scaler (q) high q shows trust in sensor measurements while low q shows trust in my physics based model. Because of this we use the process noise in equation 10 when calculating $P_{pred}$ this accounts for potential error in the model and sensor measure and creates a more reliable estimate. Q can be estimated as:

$$[12] \qquad Q = q \cdot \begin{bmatrix} \frac{dt^4}{4} & \frac{dt^3}{2} & 0 & 0 \\ \frac{dt^3}{2} & dt^2 & 0 & 0 \\ 0 & 0 & \frac{dt^4}{4} & \frac{dt^3}{2} \\ 0 & 0 & \frac{dt^3}{2} & dt^2 \end{bmatrix}$$

### 2.6 – Measurement Noise (R)

Measurement noise is a value showing how much we trust the sensors we are using. This depends on the robustness of the sensors you are using. R looks like:

$$[13] \qquad R = \begin{bmatrix} \sigma_{m_x} & 0 \\ 0 & \sigma_{m_y} \end{bmatrix}$$

Where $\sigma_{m_x}$ and $\sigma_{m_y}$ are the standard deviation of the position sensor measurement error in the x and y directions respectively.

## 3) Key Values

### 3.2 – Residual Covariance (S)

Residual covariance is based on a different value called the residual; the residual represents the error between the model's estimate and the sensor's reading. The equation is shown below where z is the altitude sensor reading.

$$[14] \qquad y = z - Hx_{pred}$$

The residual covariance comes in to represent how much we trust the residual, the error that the residual represents is distorted due to noise in the sensor as well as model deficiencies that we mentioned earlier such as drag or thrust variation. S is computed as follows:

$$[15] \qquad S = HP_{pred}H^T + R$$

This expands into:

$$[16] \qquad S = HP_{pred}H^T + R =$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} \sigma_x^2 & cov(x, v_x) & 0 & 0 \\ cov(v_x, x) & \sigma_{v_x}^2 & 0 & 0 \\ 0 & 0 & \sigma_y^2 & cov(y, v_y) \\ 0 & 0 & cov(v_y, y) & \sigma_{v_y}^2 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} \sigma_{m_x} & 0 \\ 0 & \sigma_{m_y} \end{bmatrix} =$$

$$\begin{bmatrix} \sigma_x^2 + \sigma_{m_x} & 0 \\ 0 & \sigma_y^2 + \sigma_{m_y} \end{bmatrix}$$

In its simplest form residual covariance represents the sum of model uncertainty and measurement uncertainty.

### 3.3 – Kalman Gain (K)

The Kalman Gain is the center of the Kalman Filter, it decides how heavily you weigh the sensor measurement as opposed to your model prediction. If you have an outlier sensor read the Kalman gain will adjust and more heavily favor the model's prediction and vice versa. The equation does a good job of explaining how the gain is calculated.

$$[17] \qquad K = P_{pred}H^T S^{-1}$$

Here we are taking the uncertainty of the model's prediction and dividing it by the uncertainty of the residual. This is taking model error and dividing by total error, if K is high that means that model error accounts for most of the error in the system and the filter will side with the sensor's reading and vice versa.

## 4) Error Integration

### 4.1 – Sensor Error

I applied a lot of sensor error throughout the whole project in order to test the Kalman filter. I applied errors to every facet of the project including adding acceleration, velocity, and position sensor noise. Specifically, the standard deviations of the error for each facet were 2.5 $\frac{m}{s^2}$ for acceleration, 60 $\frac{m}{s}$ for velocity, and 500 m for position. These are very high values by design this makes it easier to see the impact of the filter

### 4.2 – Model Error

The physics-based model was also meant to have a substantial amount of error, I calculated the true rocket acceleration using this equation.

$$a_{true} = \left(a_{peak} - a_{liftoff}\right) - \left(\frac{t_{current} - t_{peak}}{40}\right)^2 \quad [18]$$

I used a similar approach to calculate the model's estimated acceleration, using the following equation:

$$a_{est} = \left(a_{peak} - a_{liftoff} + mockError\right) - \left(\frac{t_{current} - t_{peak}}{30}\right)^2 \quad [19]$$

This creates a model that follows a similar acceleration curve, however there is still substantial error. The error is also increased over each integration from acceleration to velocity then velocity to position.

### 4.3 – Error Plots

These plots below show how the sensor error and model error compare to the true rocket data. This is a good way to visualize just how much work the filter needs to do to conform the data to accurately estimate the true rocket states.

4.3.1 – Acceleration Error

The acceleration error of the model is relatively low, as integration takes place the model will become less and less accurate. However, when evaluating acceleration our model is the best basis to create an accurate estimate. The sensor measurement data has a consistent amount of error throughout and does not get less accurate with integration.

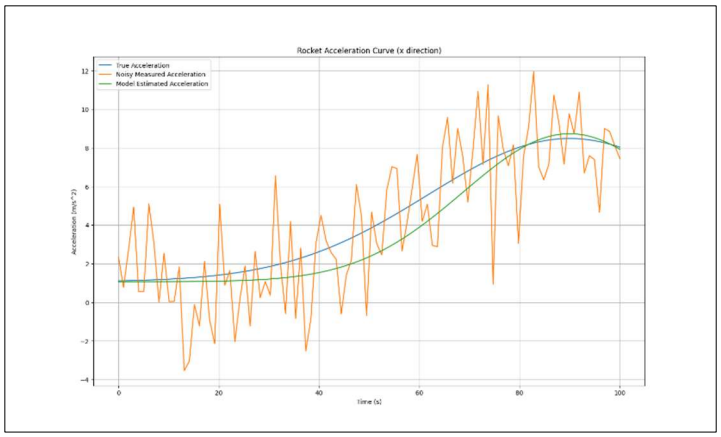


*Figure 1: Error Acceleration X*

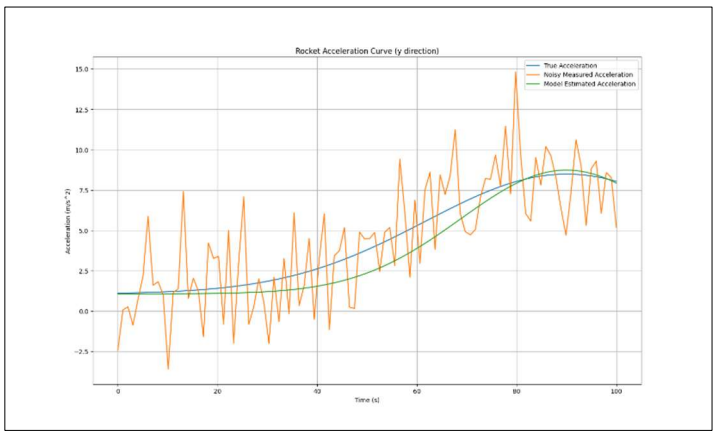


*Figure 2: Error Acceleration Y*

### 4.3.2 – Velocity Error

The magnitude of the velocity error of the model is fairly similar to that of the sensor measurement data. Because of this the Kalman filter will rely on both the sensor data and the model's data almost evenly when trying to predict the state of the rocket.

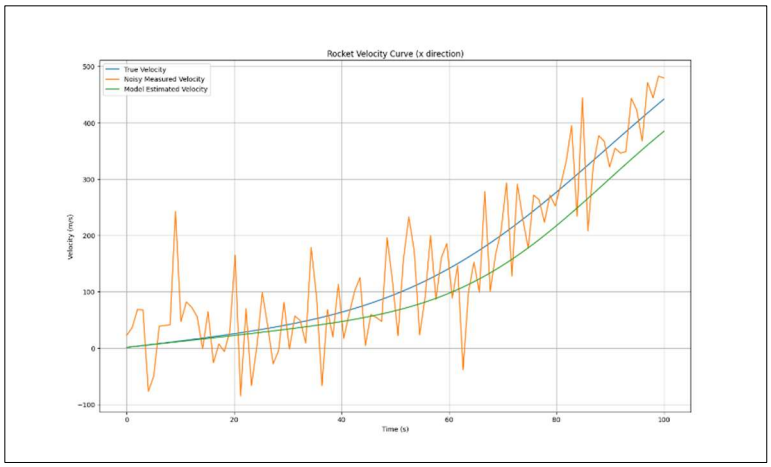


*Figure 3: Error Velocity X*

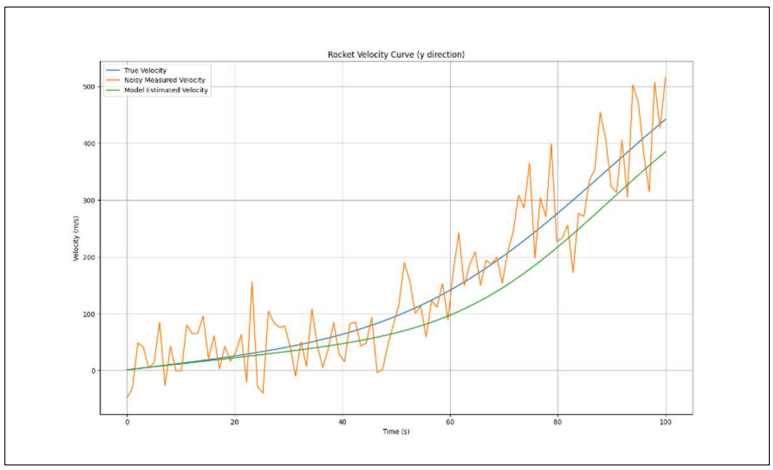


*Figure 4: Error Velocity Y*

4.3.3– Position Error

For the position data the model gets very inaccurate while the sensor data stays closer to the rocket's true position. Because of this when filtering position the filter will more heavily rely on the sensor measurement data over that of the model. As referenced earlier the model calculates acceleration and integrates to find velocity and position. Because of this with each integration the error is magnified.

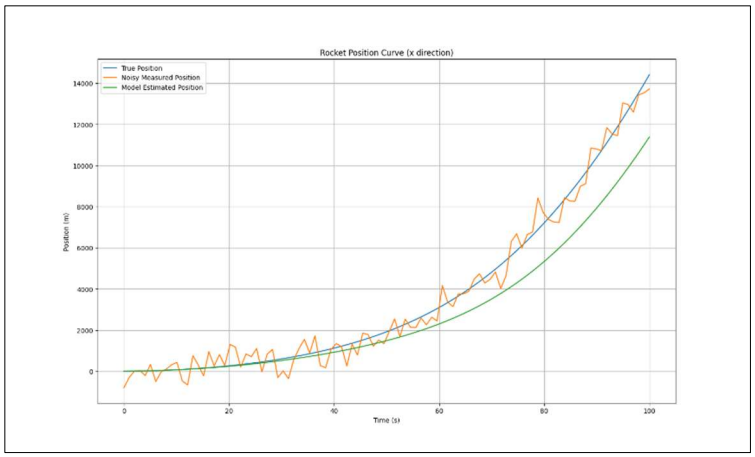


*Figure 5: Error Position X*

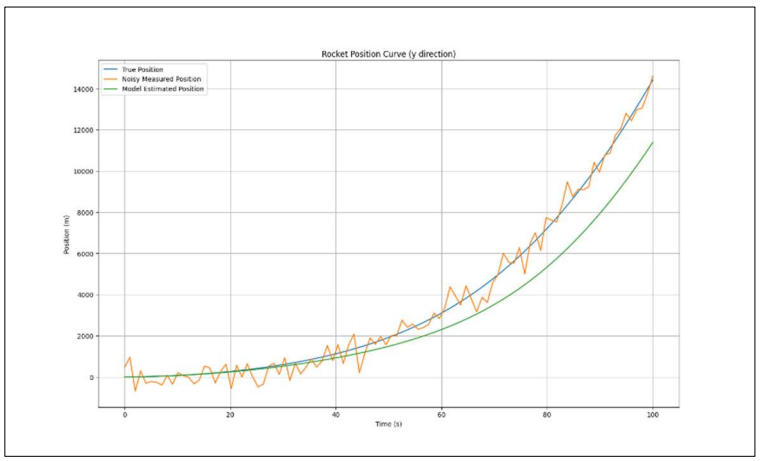


*Figure 6: Error Position Y*

# 5) Results

### 5.1 – Kalman Filter Output

The main idea of a Kalman filter is that it takes in noisy sensor measurement data and data from in this case a physics-based model. The filter then creates a state estimation of the rocket using a combination of this data. With this being the case the goal of the Kalman filter is to estimate the state of the rocket with less error than the sensor data and the model data.

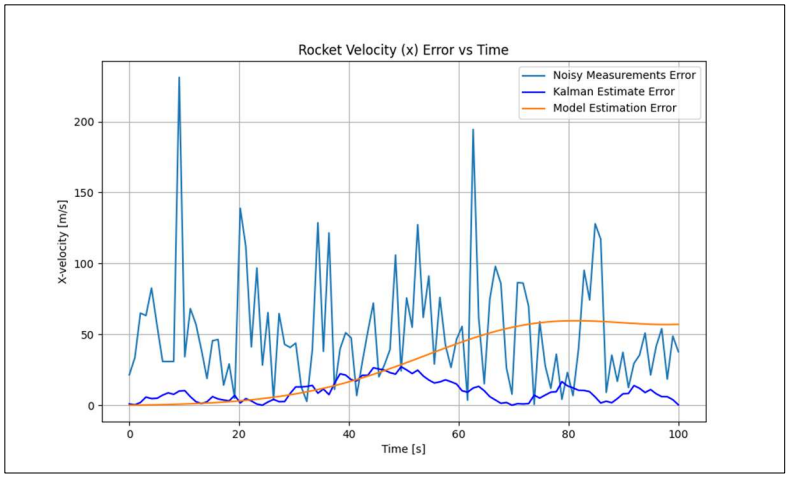


*Figure 7: Error Comparison Velocity X*

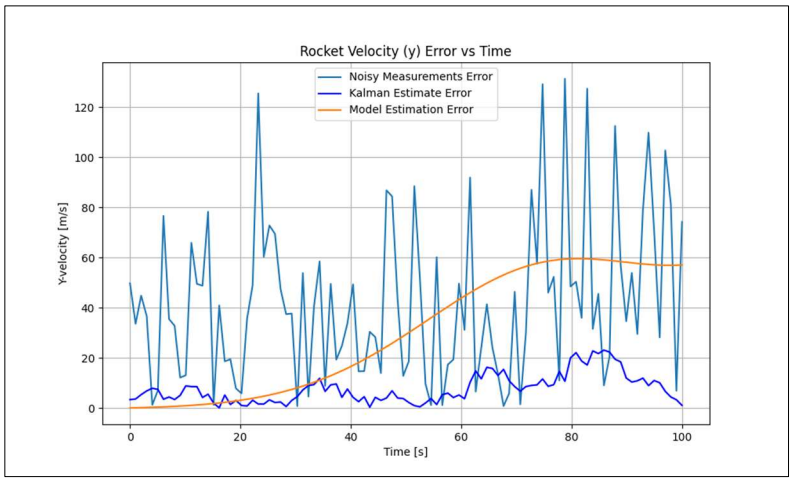


*Figure 8: Error Comparison Velocity Y*

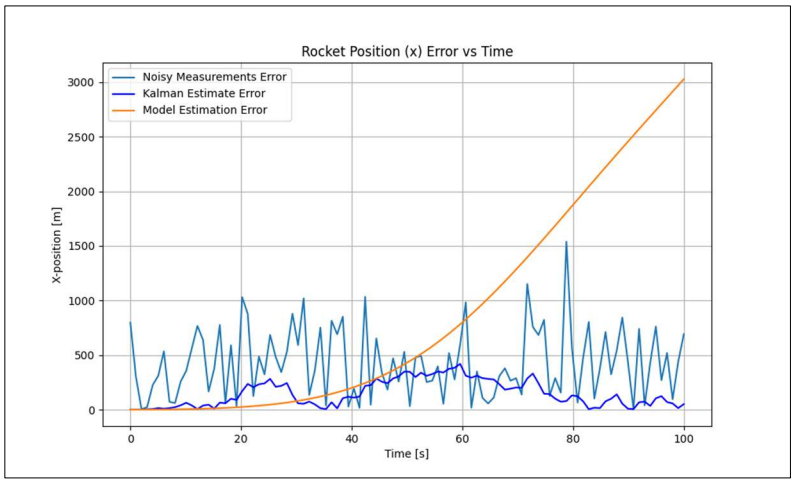


*Figure 9: Error Comparison Position X*

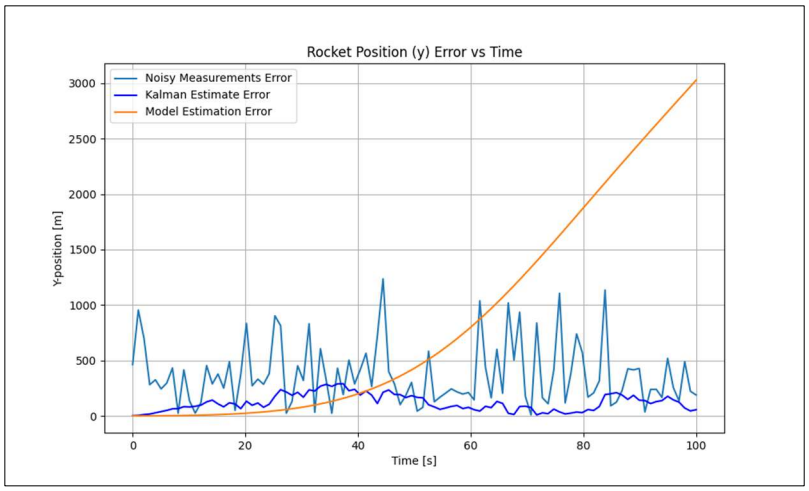


*Figure 10: Error Comparison Position Y*

These plots show that the Kalman filter output is more accurate than both the sensor measured data and the model predicted data. This means that the filter is a success. Even with extremely high magnitudes of error from the sensors and model, the filter is still within 500 meters of the rocket's position at all times and within 25 meters per second of its velocity. With realistic sensor and model data the filter would be able to be used in high leverage space missions that rely on precision and accuracy from their filtering system.

# 6) Appendix/Code

### 6.1 – rocketKalman.py (main and only python file)

```
### Riley Martin
# Final Project

# rocket_kalman.py

import numpy as np
import matplotlib.pyplot as plt
import os

# ---------------------------------------- Write out Function ----------------------------------------

def write_sensor_file(folder, filename, x_data, y_data, x_header, y_header):
```

```
    # Full file path
    filepath = os.path.join(folder, filename)

    with open(filepath, "w") as file:
        file.write(f"{x_header}, {y_header}\n")

        for x, y in zip(x_data, y_data):
            file.write(f"{x}, {y}\n")

# ---------------------------------------- Simulate rocket flight ----------------------------------------
# This section is for modeling the flight dynamics of the rocket

# Define theta as the launch angle of the rocket (constant)
theta = np.pi/4

# Define the length of the simulation and create time step (dt)
t_final = 100  # time of first burn (s)
t = np.linspace(0, t_final, 100)
dt = t_final / len(t)

# Acceleration parameters of a typical rocket (average stats)
a_liftoff = 1.5   # accel at liftoff (m/s^2)
a_peak = 12.0      # peak accel (m/s^2)
t_peak = 90     # time of peak accel (s)

# Calculate the true acceleration (This is where the rocket really is in both x and y space)
a_true = a_liftoff + (a_peak - a_liftoff) * np.exp(-((t - t_peak)/40)**2)
a_true_x = a_true * np.cos(theta)
a_true_y = a_true * np.sin(theta)

# Calculate the true velocity (This is the rocket's actual velocity in both x and y)
v_true = np.cumsum(a_true*dt)
v_true_x = v_true * np.cos(theta)
v_true_y = v_true * np.sin(theta)
```

```
# Calculate the true position (This is the rocket's actual position in x and y space)

p_true = np.cumsum(v_true*dt)

p_true_x = p_true * np.cos(theta)

p_true_y = p_true * np.sin(theta)


# ---------------------------------------- Create the Calculated Model ----------------------------------------

# This model is meant to semi-accuratly model the roacket's acceleration, velocity, and position. While still having error for the filter to correct


# Compute the mock calculated acceleration (This is a physics based estimation of the rocket's acceleration) (Error is added intentionally)

mock_error = 0.35

a_est = a_liftoff + (a_peak - a_liftoff + mock_error) * np.exp(-((t - t_peak)/30)**2)

a_est_x = a_est * np.cos(theta)

a_est_y = a_est * np.sin(theta)


# Compute the mock calculated velocity (This is a physics based estimation of the rocket's velocity) (Error is still present from acceleration calc)

v_est = np.cumsum(a_est*dt)

v_est_x = v_est * np.cos(theta)

v_est_y = v_est * np.sin(theta)


# Compute the mock calculated position (This is a physics based estimation of the rocket's position) (Error is still present from acceleration calc)

p_est = np.cumsum(v_est*dt)

p_est_x = p_est * np.cos(theta)

p_est_y = p_est * np.sin(theta)


# ---------------------------------------- Add measurement noise ----------------------------------------

# This section is created to add noise to the senor data. The sensors should be based off the true data but should not be exact. This error will be

# corrected by the filter


# Set a randomizer seed for reproducibility while still getting seemingly randomized data

np.random.seed(42)
```

```
# Set the standard deviation of both acceleration and position error. This is how the error magnitude is set
accel_noise_std = 2.5
velo_noise_std = 60
pos_noise_std = 500

# Now randomize the acceleration measured data this is simulating noise
accel_meas_x = np.random.normal(a_true_x, accel_noise_std, size=len(t))
accel_meas_y = np.random.normal(a_true_y, accel_noise_std, size=len(t))

# Now randomize the acceleration measured data this is simulating noise
velo_meas_x = np.random.normal(v_true_x, velo_noise_std, size=len(t))
velo_meas_y = np.random.normal(v_true_y, velo_noise_std, size=len(t))

# Do the same for position
pos_meas_x = np.random.normal(p_true_x, pos_noise_std, size=len(t))
pos_meas_y = np.random.normal(p_true_y, pos_noise_std, size=len(t))

# ---------------------------------------- Write out Data ----------------------------------------
# This section uses the write out function to write data to files. Files will be included in submission

# Write out sensor measurement data
write_sensor_file("data", "measures_accel.txt", accel_meas_x, accel_meas_y,"accel_x", "accel_y")
write_sensor_file("data", "measures_velo.txt", velo_meas_x, velo_meas_y,"velo_x", "velo_y")
write_sensor_file("data", "measures_pos.txt", pos_meas_x, pos_meas_y,"pos_x", "pos_y")

# Write out the model estimated data
write_sensor_file("data", "estimated_accel.txt", a_est_x, a_est_y,"accel_x", "accel_y")
write_sensor_file("data", "estimated_velo.txt", v_est_x, v_est_y,"velo_x", "velo_y")
write_sensor_file("data", "estimated_pos.txt", p_est_x, p_est_y,"pos_x", "pos_y")

# Wite out the rocket's true data
write_sensor_file("data", "true_accel.txt", a_true_x, a_true_y,"accel_x", "accel_y")
write_sensor_file("data", "true_velo.txt", v_true_x, v_true_y,"velo_x", "velo_y")
write_sensor_file("data", "true_pos.txt", p_true_x, p_true_y,"pos_x", "pos_y")
```

```
# ---------------------------------------- Plot the acceleration error in x and y ----------------------------------------

# Here is where acceleration plots are made for true data, noisy sensor readings, and model estimation. This is a good way to see error before integration

# and filtering


# Plot acceleration in the x direction (True data, noisy sensor readings, and model estimation)

plt.figure()

plt.plot(t, a_true_x, label='True Acceleration')

plt.plot(t, (accel_meas_x), label='Noisy Measured Acceleration')

plt.plot(t, (a_est_x), label='Model Estimated Acceleration')

plt.xlabel('Time (s)')

plt.ylabel('Acceleration (m/s^2)')

plt.title('Rocket Acceleration Curve (x direction)')

plt.grid(True)

plt.legend()

plt.show()


# Plot acceleration in the y direction (True data, noisy sensor readings, and model estimation)

plt.figure()

plt.plot(t, a_true_y, label='True Acceleration')

plt.plot(t, (accel_meas_y), label='Noisy Measured Acceleration')

plt.plot(t, (a_est_y), label='Model Estimated Acceleration')

plt.xlabel('Time (s)')

plt.ylabel('Acceleration (m/s^2)')

plt.title('Rocket Acceleration Curve (y direction)')

plt.grid(True)

plt.legend()

plt.show()


# ---------------------------------------- Plot the velocity error in x and y ----------------------------------------

# Here is where velocity plots are made for true data, noisy sensor readings, and model estimation. This is a good way to see error after integration

# and before filtering


# Plot velocity in the x direction (True data, noisy sensor readings, and model estimation)

plt.figure()
```

```
plt.plot(t, v_true_x, label='True Velocity')
plt.plot(t, (velo_meas_x), label='Noisy Measured Velocity')
plt.plot(t, (v_est_x), label='Model Estimated Velocity')
plt.xlabel('Time (s)')
plt.ylabel('Velocity (m/s)')
plt.title('Rocket Velocity Curve (x direction)')
plt.grid(True)
plt.legend()
plt.show()

# Plot velocity in the y direction (True data, noisy sensor readings, and model estimation)
plt.figure()
plt.plot(t, v_true_y, label='True Velocity')
plt.plot(t, (velo_meas_y), label='Noisy Measured Velocity')
plt.plot(t, (v_est_y), label='Model Estimated Velocity')
plt.xlabel('Time (s)')
plt.ylabel('Velocity (m/s)')
plt.title('Rocket Velocity Curve (y direction)')
plt.grid(True)
plt.legend()
plt.show()

# ---------------------------------------- Plot the position error in x and y ----------------------------------------
# Here is where position plots are made for true data, noisy sensor readings, and model estimation. This is a good way to see error after integration and
# before filtering

# Plot position in the x direction (True data, noisy sensor readings, and model estimation)
plt.figure()
plt.plot(t, p_true_x, label='True Position')
plt.plot(t, (pos_meas_x), label='Noisy Measured Position')
plt.plot(t, (p_est_x), label='Model Estimated Position')
plt.xlabel('Time (s)')
plt.ylabel('Position (m)')
plt.title('Rocket Position Curve (x direction)')
plt.grid(True)
```

```
plt.legend()
plt.show()

# Plot position in the y direction (True data, noisy sensor readings, and model estimation)
plt.figure()
plt.plot(t, p_true_y, label='True Position')
plt.plot(t, (pos_meas_y), label='Noisy Measured Position')
plt.plot(t, (p_est_y), label='Model Estimated Position')
plt.xlabel('Time (s)')
plt.ylabel('Position (m)')
plt.title('Rocket Position Curve (y direction)')
plt.grid(True)
plt.legend()
plt.show()

# ---------------------------------------- Kalman Filter Setup ----------------------------------------
# This step is setting up all matricies that are used in kalman filtering. This is set up before running the filter in the loop

# Set initial states (everything starts at 0)
# This vector is consitant throughout(x_pos, x_vel, y_pos, y_vel)
x = np.array([[0], [0], [0], [0]])

# State transition matrix
# Position is iterated by pos_old + v * dt
# While velocity is assumed constant in this step
F = np.array([[1, dt, 0, 0], [0, 1, 0, 0], [0, 0, 1, dt], [0, 0, 0, 1]])

# Control input matrix
# Position
B = np.array([[0.5*dt**2, 0], [dt, 0], [0, 0.5*dt**2], [0, dt]])

# Observation matrix (we measure altitude directly)
H = np.array([[1, 0, 0, 0], [0, 0, 1, 0]])

# Covariance matrices
```

```
P = np.eye(4) * 100  # initial estimate covariance


q = 5.0 # This is a scaler high q shows trust in sensor measurements while low q shows trust in the model

Q = np.array([[dt**4/4, dt**3/2, 0, 0],[dt**3/2, dt**2, 0, 0],[0, 0, dt**4/4, dt**3/2],[0, 0, dt**3/2, dt**2]]) * q


R = np.array([[pos_noise_std**2, 0],[0, pos_noise_std**2]])


# Allocate storage for estimates

x_estimates = np.zeros((4, len(t)))


# ---------------------------------------- Run Kalman Filter ----------------------------------------

# This is where the kalman filter is run. It is run at each timestep and it takes in both the physics based model and the noisy sensor data.

# It uses both of these values to find a middle ground estimation of where the rocket is at each timestep.


for k in range(len(t)):

    # Prediction step

    u = np.array([[accel_meas_x[k]], [accel_meas_y[k]]]) # acceleration measurement as control input

    x_pred = F @ x + B @ u

    P_pred = F @ P @ F.T + Q

    z = np.array([[pos_meas_x[k]], [pos_meas_y[k]]])


    # Update step

    u = np.array([[accel_meas_x[k]], [accel_meas_y[k]]])

    y = z - H @ x_pred

    S = H @ P_pred @ H.T + R

    K = P_pred @ H.T @ np.linalg.inv(S)


    x = x_pred + K @ y

    P = (np.eye(4) - K @ H) @ P_pred


    x_estimates[:, k] = x.flatten()


# ---------------------------------------- Plot the velocity in x and y ----------------------------------------

# Here is where velocity plots are made for true data, noisy sensor readings, model estimation, and kalman filter output. This is a good way to see
```

```
# error after one round of integration (acceleration > velocity) and filtering.

# X-velocity
plt.figure(figsize=(10,6))
plt.plot(t, v_true_x, 'k-', label='True Velocity')
plt.plot(t, x_estimates[1,:], 'b-', label='Estimated Velocity')
plt.xlabel('Time [s]')
plt.ylabel('Velocity (x) [m/s]')
plt.title('Rocket Velocity (x) Estimation vs Time')
plt.legend()
plt.grid(True)
plt.show()

# Y-velocity
plt.figure(figsize=(10,6))
plt.plot(t, v_true_y, 'k-', label='True Velocity')
plt.plot(t, x_estimates[3,:], 'b-', label='Estimated Velocity')
plt.xlabel('Time [s]')
plt.ylabel('Velocity (y) [m/s]')
plt.title('Rocket Velocity (y) Estimation vs Time')
plt.legend()
plt.grid(True)
plt.show()

# ---------------------------------------- Plot the position in x and y ----------------------------------------
# Here is where position plots are made for true data, noisy sensor readings, and model estimation. This is a good way to see error after two rounds of
# integration (acceleration > velocity > position) and filtering.

# Plot position in the x direction (True data, noisy sensor readings, model estimation, and kalman filter output)
plt.figure(figsize=(10,6))
plt.plot(t, p_true_x, 'k-', label='True X-position')
plt.plot(t, pos_meas_x, 'r.', label='Noisy Measurements')
plt.plot(t, p_est_x, label='Model Estimation')
plt.plot(t, x_estimates[0,:], 'b-', label='Kalman Estimate')
plt.xlabel('Time [s]')
```

```
plt.ylabel('X-position [m]')
plt.title('Rocket Position (x) Estimation vs Time')
plt.legend()
plt.grid(True)
plt.show()

# Plot position in the y direction (altitude) (True data, noisy sensor readings, model estimation, and kalman filter output)
plt.figure(figsize=(10,6))
plt.plot(t, p_true_y, 'k-', label='True Altitude')
plt.plot(t, pos_meas_y, 'r.', label='Noisy Measurements')
plt.plot(t, p_est_y, label='Model Estimation')
plt.plot(t, x_estimates[2,:], 'b-', label='Kalman Estimate')
plt.xlabel('Time [s]')
plt.ylabel('Y-position [m]')
plt.title('Rocket Position (y) Estimation vs Time')
plt.legend()
plt.grid(True)
plt.show()

# ---------------------------------------- Plot the error of both the velocity and position in x and y ----------------------------------------
# These plots are used to evaluate the effectiveness of the kalman filter. The filter's error should be less than the sensor data and the model that
# is how success is judged.

# Plot error in the X-direction velocity (True data, noisy sensor readings, model estimation, and kalman filter output)
plt.figure(figsize=(10,6))
plt.plot(t, np.abs(velo_meas_x-v_true_x), label='Noisy Measurements Error')
plt.plot(t, np.abs(x_estimates[1,:]-v_true_x), 'b-', label='Kalman Estimate Error')
plt.plot(t, np.abs(v_est_x-v_true_x), label='Model Estimation Error')
plt.xlabel('Time [s]')
plt.ylabel('X-velocity [m/s]')
plt.title('Rocket Velocity (x) Error vs Time')
plt.legend()
plt.grid(True)
```

```
plt.show()

# Plot error in the Y-direction velocity (True data, noisy sensor readings, model estimation, and kalman filter output)
plt.figure(figsize=(10,6))
plt.plot(t, np.abs(velo_meas_y-v_true_y), label='Noisy Measurements Error')
plt.plot(t, np.abs(x_estimates[3,:]-v_true_y), 'b-', label='Kalman Estimate Error')
plt.plot(t, np.abs(v_est_y-v_true_y), label='Model Estimation Error')
plt.xlabel('Time [s]')
plt.ylabel('Y-velocity [m/s]')
plt.title('Rocket Velocity (y) Error vs Time')
plt.legend()
plt.grid(True)
plt.show()

# Plot error in the X-direction position (True data, noisy sensor readings, model estimation, and kalman filter output)
plt.figure(figsize=(10,6))
plt.plot(t, np.abs(pos_meas_x-p_true_x), label='Noisy Measurements Error')
plt.plot(t, np.abs(x_estimates[0,:]-p_true_x), 'b-', label='Kalman Estimate Error')
plt.plot(t, np.abs(p_est_x-p_true_x), label='Model Estimation Error')
plt.xlabel('Time [s]')
plt.ylabel('X-position [m]')
plt.title('Rocket Position (x) Error vs Time')
plt.legend()
plt.grid(True)
plt.show()

# Plot error in the Y-direction position (altitude) (True data, noisy sensor readings, model estimation, and kalman filter output)
plt.figure(figsize=(10,6))
plt.plot(t, np.abs(pos_meas_y-p_true_y), label='Noisy Measurements Error')
plt.plot(t, np.abs(x_estimates[2,:]-p_true_y), 'b-', label='Kalman Estimate Error')
plt.plot(t, np.abs(p_est_y-p_true_y), label='Model Estimation Error')
plt.xlabel('Time [s]')
plt.ylabel('Y-position [m]')
plt.title('Rocket Position (y) Error vs Time')
```

```
plt.legend()
plt.grid(True)
plt.show()
```